

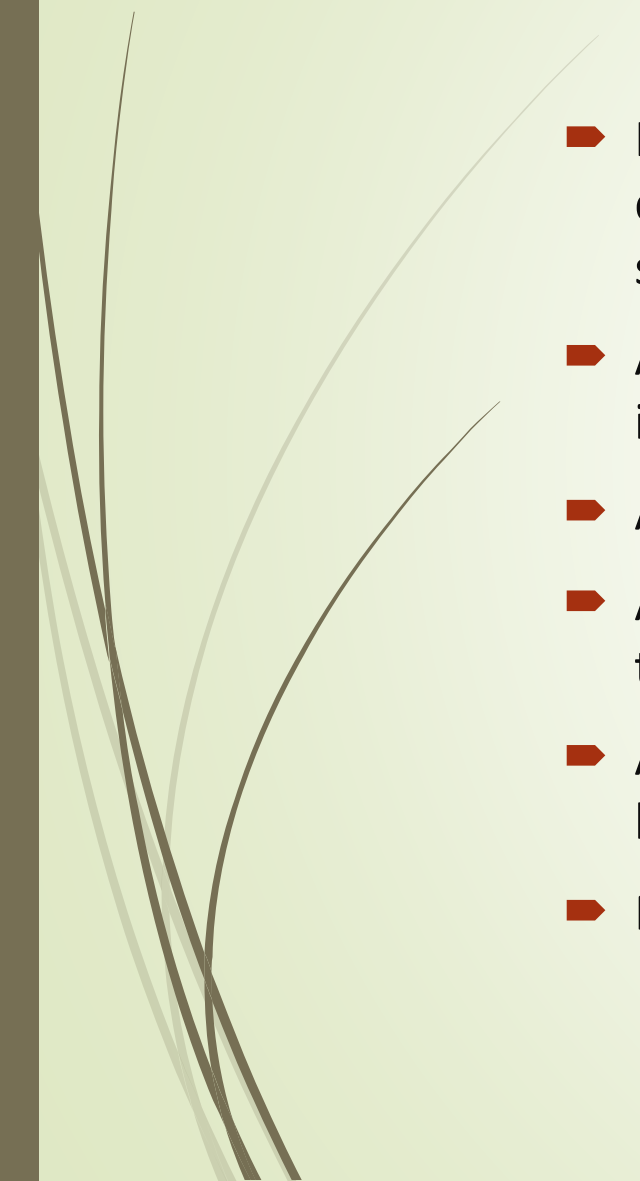
DATA STRUCTURES

Complexity Theory-I





Algorithms



- In mathematics, computer science, and related subjects, an algorithm can be defined as “a process that performs some sequence of operations in order to solve a given problem”.
- Algorithm is a well-defined computational procedure that takes some value (s) as input and produces some value (s) as output.
- Algorithms are used for calculation, data processing, and many other fields.
- A good algorithm is like using the right tool in the workshop. It does the job with the right amount of effort.
- Algorithms can be expressed in many different notations, including natural languages, pseudo-code, flowcharts and programming languages.
- Program = Algorithms + Data Structures

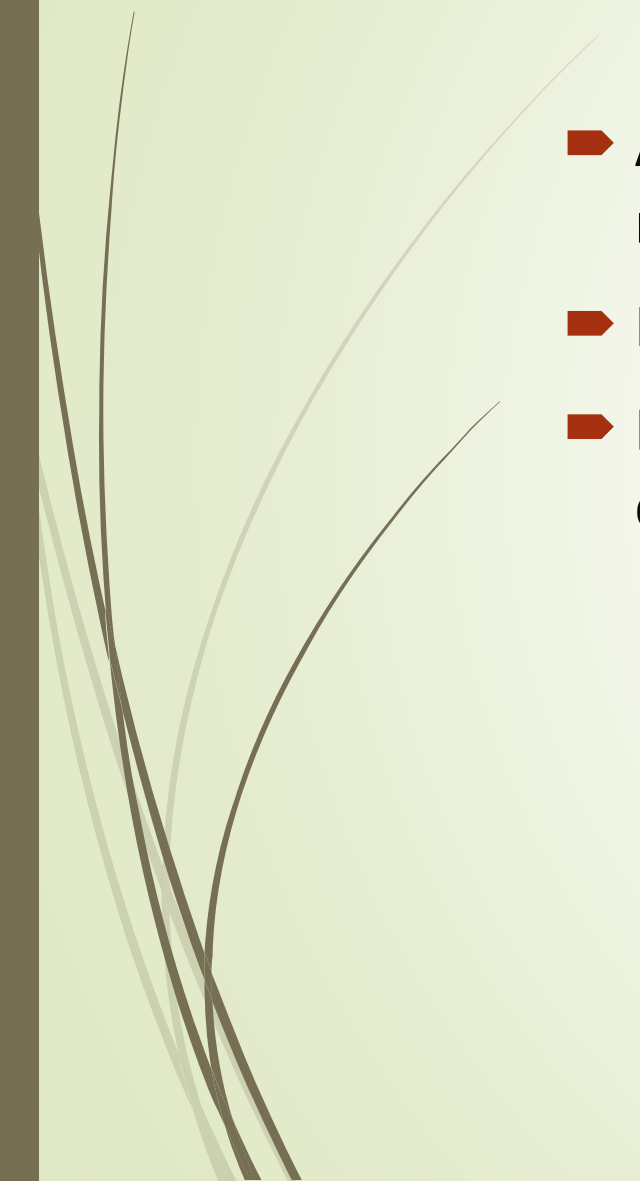


Analysis of Algorithms

- Just writing a syntax-error-free program is not enough.
- We need to know whether the algorithm is correct or not, i.e., whether it can give correct answers for the inputs.
- If the program is run on a large data set, the running time becomes an issue.
- We want to know how the algorithm performs when the input size is large.
- The program may be computationally inefficient, or may need lots of memory
- We analysis algorithms, rather than problems. A problem can be solved with several algorithms, some are more efficient than others.



Analysis of Algorithms

- Analysis of Algorithm refers to calculating or guessing resources needful for the algorithm.
 - Resources means computer memory, processing time etc.
 - In all these factors, time is most important because the program developed should be fast enough.
- 

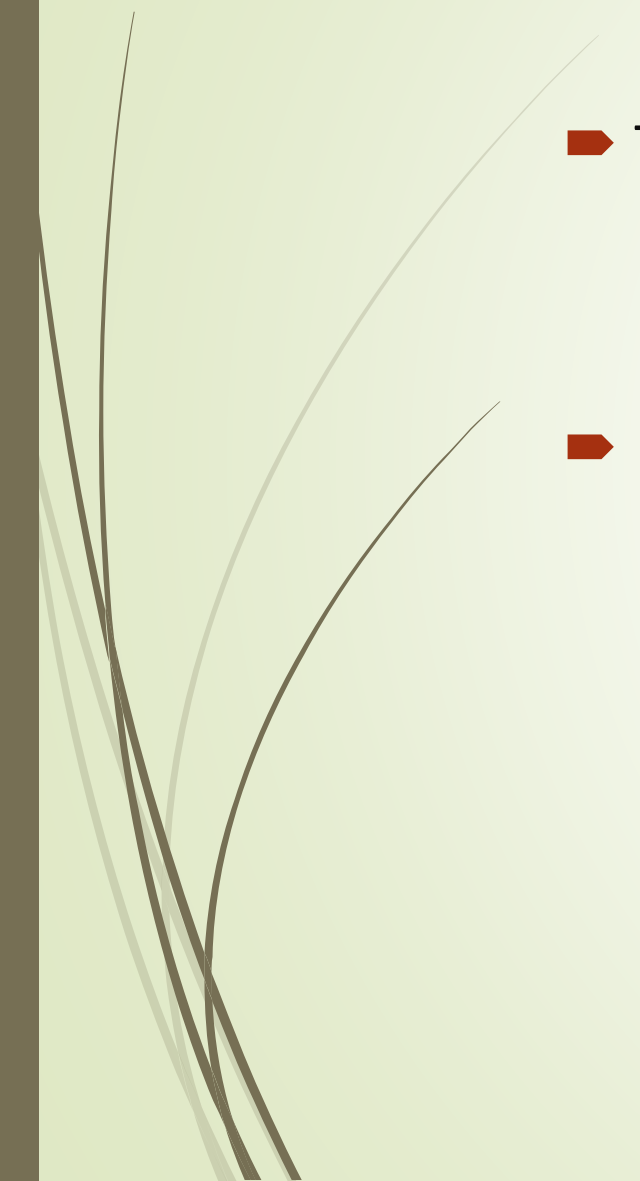


Analysis of Algorithms

- Given two or more algorithms to solve the same problem, how do we select the best one?
- Some criteria for selecting an algorithm
 - Is it easy to implement, understand, modify?
 - How long does it take to run it to completion?
 - How much of computer memory does it use?
- Software engineering is primarily concerned with the first criteria
- In this course we are interested in the second and third criteria



Analysis of Algorithms

- ▶ Time complexity
 - ▶ The amount of time that an algorithm needs to run to completion
 - ▶ Space complexity
 - ▶ The amount of memory an algorithm needs to run
- 



Analysis of Algorithms

- There are several factors affecting the running time
 - computer
 - compiler
 - algorithm
 - input to the algorithm
 - *The content of the input affects the running time typically, the input size (number of items in the input) is the main consideration*
 - E.g. sorting problem $\Rightarrow$ the number of items to be sorted.

Running time of an Algorithm

- ▶ Running time of an algorithm depends upon the input size and nature of input.
- ▶ Most algorithms transform input objects into output objects



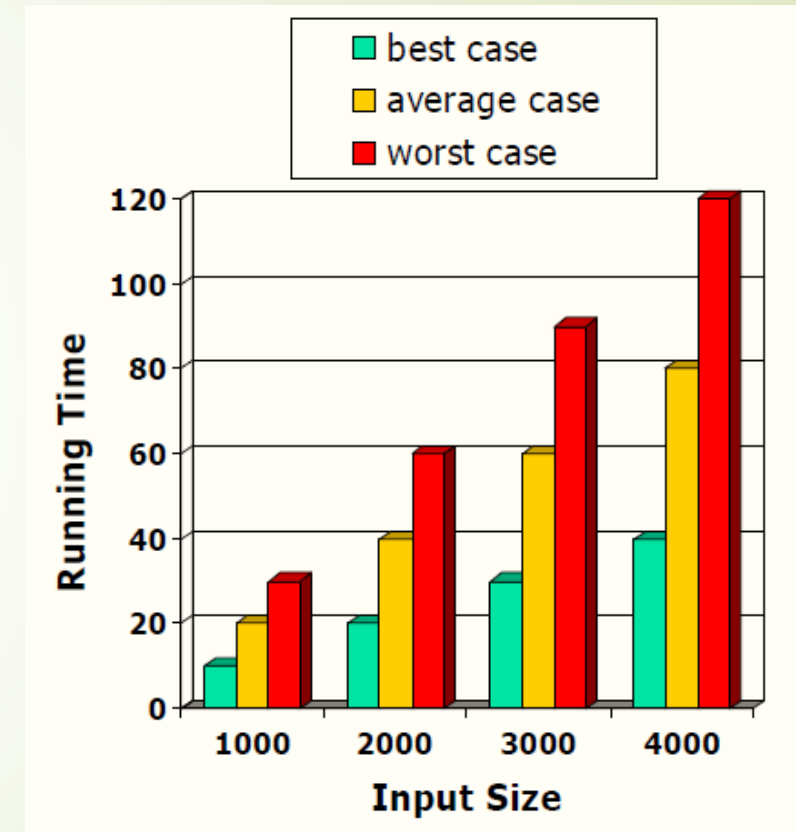
- ▶ The running time of an algorithm typically grows with the input size
 - ▶ idea: analyze running time as a function of input size

Run Time of an Algorithm

- Most important factor affecting running time is usually the size of the input
- Even on inputs of the same size, running time can be very different
 - Example: algorithm that finds the first prime number in an array by scanning it left to right
- Idea: Analyze running time for different cases
 - Best case
 - Worst case
 - Average case

Finding Running Time of an Algorithm

- Best case running time is usually not very useful
- Average case time is very useful but often hard to determine
- Worst case running time is easier to analyze
 - – Crucial for real-time applications such as games, finance and robotics

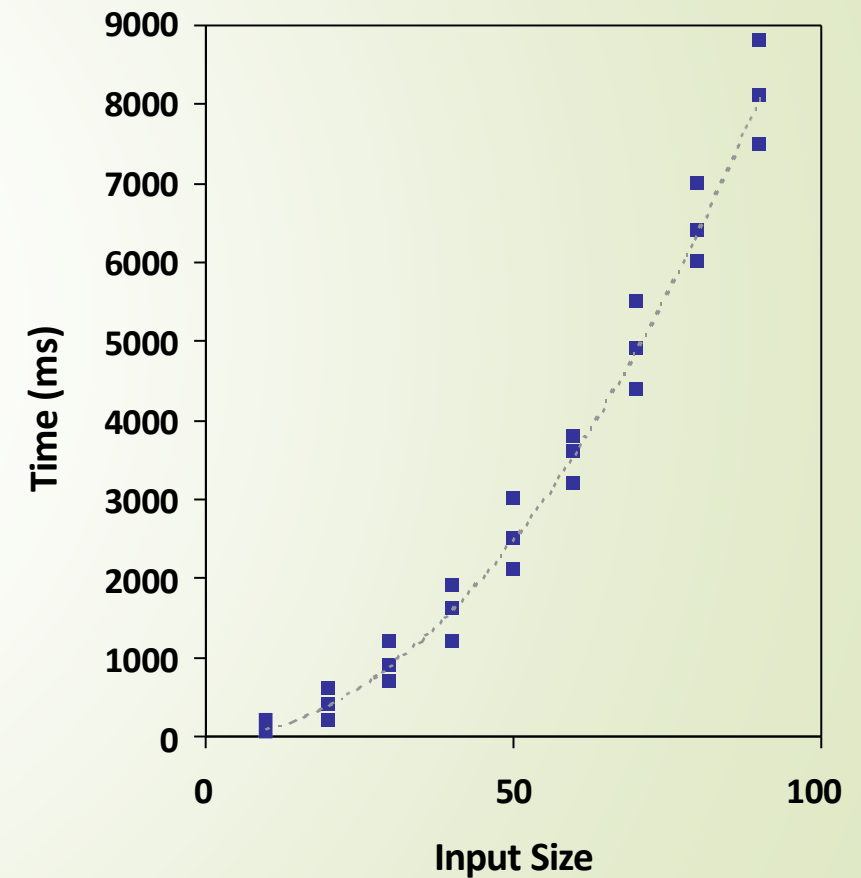


Finding Running Time of an Algorithm

- ▶ Running time is measured by number of steps/primitive operations performed
- ▶ We use RAM (Random Access Model), in which each operation (e.g. $+$, $-$, $\times$, $/$, $=$) and each memory access take one run-time unit. Loops and functions can take multiple time units.
- ▶ Count the number of basic operations of an algorithm
 - ▶ Read, write, compare, assign, jump, arithmetic operations (increment, decrement, add, subtract, multiply, divide), open, close, logical operations (not/complement, AND, OR), ...

Experimental Evaluation of Run Time

- Write a program implementing the algorithm
- Run the program with inputs of varying size
- Use clock methods to get an accurate measure of the actual running time
- Plot the results





Limitations Of Experiments

- Experimental evaluation of running time is very useful but
 - It is necessary to implement the algorithm, which may be difficult
 - Results may not be indicative of the running time on other inputs not included in the experiment
 - In order to compare two algorithms, the same hardware and software environments must be used



Theoretical Analysis of Run Time

- Uses a pseudo-code description of the algorithm instead of an implementation
 - Characterizes running time as a function of the input size n
 - Takes into account all possible inputs
 - Allows us to evaluate the speed of an algorithm independent of the hardware/software environment
- 



Analyzing an Algorithm – Operations

- Each machine instruction is executed in a fixed number of cycles
 - We may assume each operation requires a fixed number of cycles
- Idea: Use abstract machine that uses steps of time instead of secs
 - Each elementary operation takes 1 steps
- Example of operations
 - Retrieving/storing variables from memory
 - Variable assignment
 - Integer operations
 - Logical operations
 - Bitwise operations
 - Relational operations
 - Memory allocation and deallocation

Analyzing an Algorithm – Blocks of Operations

- Each operation runs in a step of 1 time unit
- Therefore any fixed number of operations also run in 1 time step
 - $s_1; s_2; \dots; s_k$
 - As long as number of operations k is constant

Example

```
int sum(int n)
{
    int partialSum;

    1  partialSum=0;           1
    2  for (int i=1;i<=n;i++)  2N+2 (=1 + (N+1) + N)
    3      partialSum += i*i*i; 4N
    4  return partialSum;      1
}
```

- ▶ Lines 1 and 4 count for one unit each.
- ▶ Line 3: executed N times, each time four units.
- ▶ Line 2: (1 for initialization, $N+1$ for all tests, N for all increments) total $2N + 2$.
- ▶ total time units: $6N + 4$

Analyzing an Algorithm

```
// Input: int A[N], array of N integers  
// Output: Sum of all numbers in array A
```

```
int SumArray(int A[], int n){  
    int s=0; ← ①  
  
    for (int i=0; i<n; i++)  
        ② → i=0; ③ ← i<n; ④ ← i++;  
        ⑤ → s = s + A[i];  
        ⑥ ← A[i]; ⑦ ← s + A[i];  
    return s; ⑧  
}
```

- ▶ Operations 1,2,8 are executed once
- ▶ Operations 4,5,6,7: Once per each iteration of for loop n iteration
- ▶ Operation 3 is executed n+1 times
- ▶ The complexity function of the algorithm is : $T(n) = 5n + 4$

Analyzing an Algorithm – Growth Rate

- Estimated running time for different values of n :
 - $n = 10 \Rightarrow 54$ steps
 - $n = 100 \Rightarrow 504$ steps
 - $n = 1,000 \Rightarrow 5004$ steps
 - $n = 1,000,000 \Rightarrow 5000004$ steps
- As n grows, number of steps $T(n)$ grow in linear proportion to n



Types of Algorithms in Relation to Time

➤ Linear Algorithm

➤ Takes a time in order of n (n is size of instances)

➤ Quadratic Algorithm

➤ Takes a time in order of n^2

➤ Cubic Algorithm

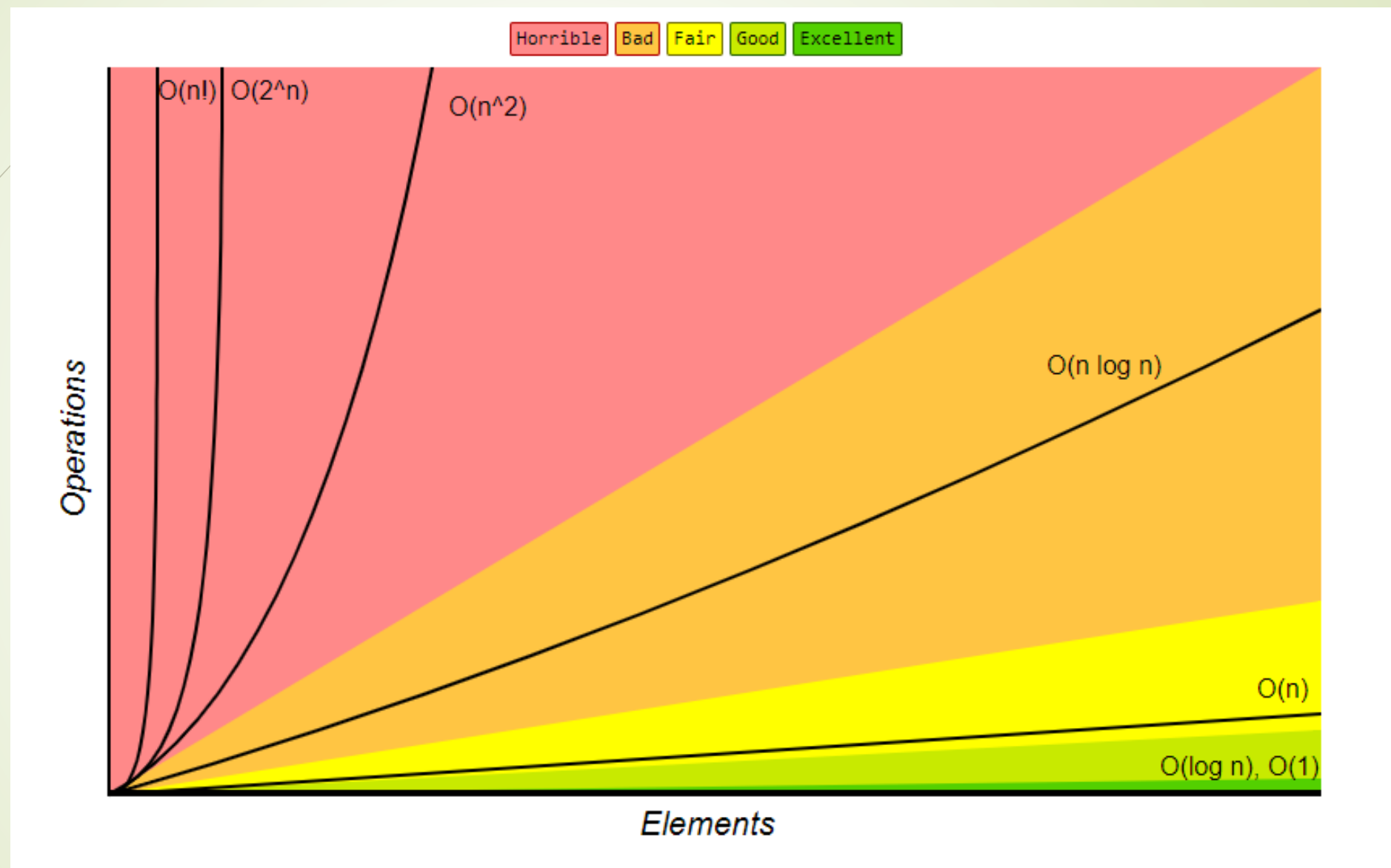
➤ Takes a time in order of n^3

➤ Polynomial Algorithm

➤ Takes a time in order of n^k (k is constant)

➤ Exponential Algorithm

➤ Takes a time in order of c^n (c is constant)





Growth Rate

- Changing the hardware/software environment
 - Affects $T(n)$ by a constant factor, but
 - Does not alter the growth rate of $T(n)$
- Thus we focus on the big-picture which is the growth rate of an algorithm
- The linear growth rate of the running time $T(n)$ is an fundamental property of algorithm `sumArray`

Constant Factors

- ▶ The growth rate is not affected by
 - ▶ Constant factors or
 - ▶ Lower-order terms
- ▶ Examples
 - ▶ $10^2n + 10^5$ is a linear function
 - ▶ $10^5n^2 + 10^8n$ is a quadratic function

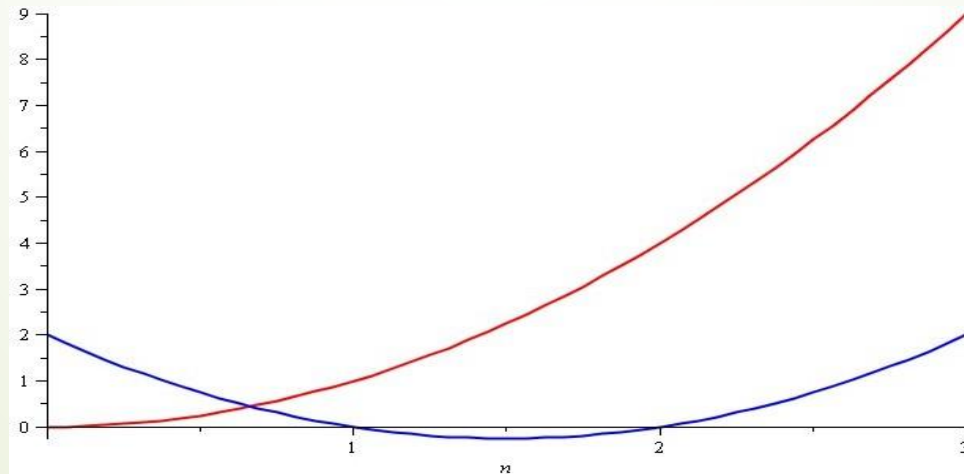
Growth Rate – Example

- Consider the two functions

$$f(n) = n^2$$

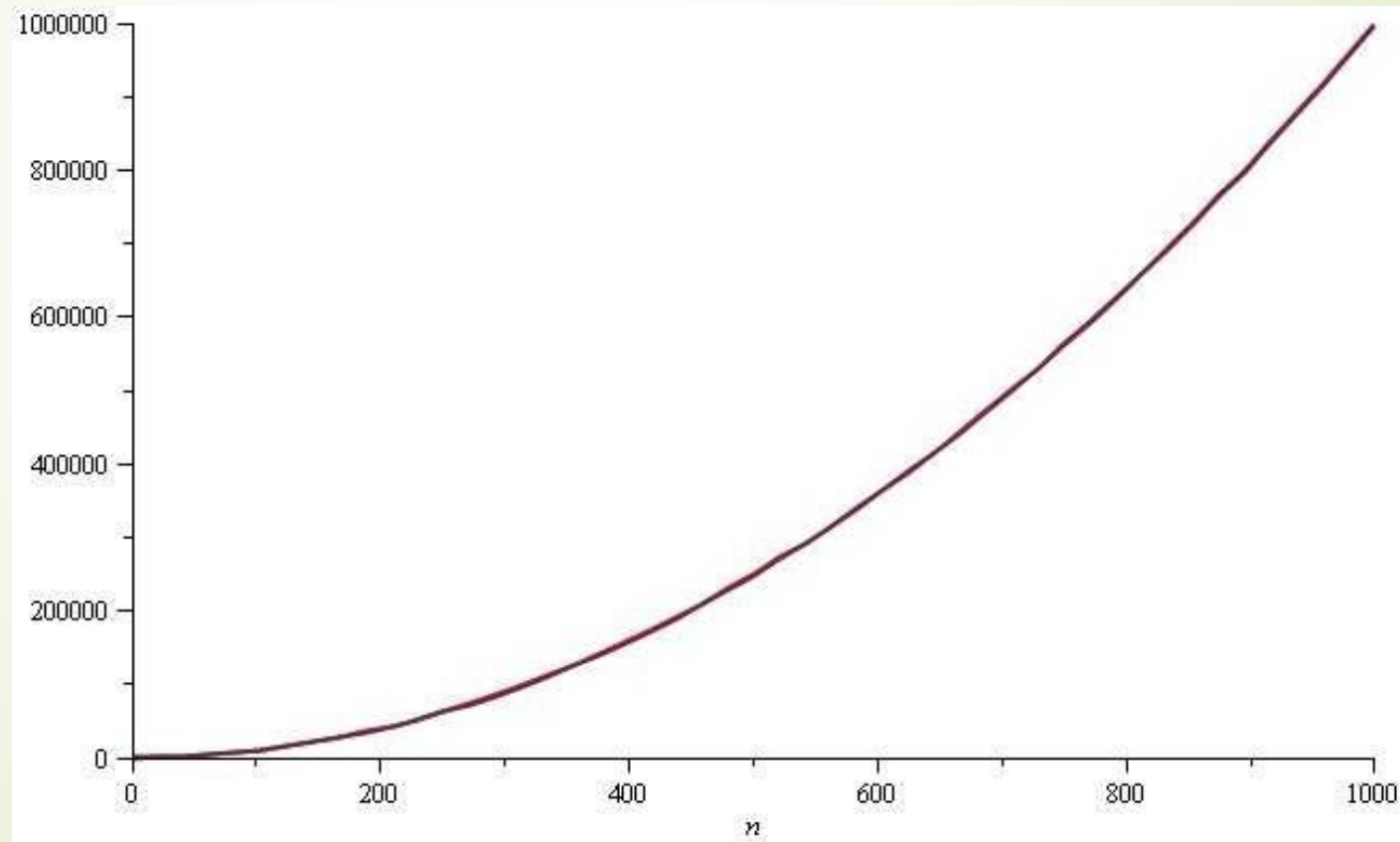
$$g(n) = n^2 - 3n + 2$$

- Around $n = 0$, they look very different



Growth Rate – Example

- Yet on the range $n = [0, 1000]$, $f(n)$ and $g(n)$ are (relatively) indistinguishable



Growth Rate – Example

- The absolute difference is large, for example,
 - $f(1000)=1000000$
 - $g(1000)=997002$
- But the relative difference is very small

$$\left| \frac{f(1000) - g(1000)}{f(1000)} \right| = 0.002998 < 0.3\%$$

- The difference goes to zero as $n \rightarrow \infty$



Constant Factors

- ▶ How do we get rid of the constant factors to focus on the essential part of the running time?
 - ▶ Asymptotic Analysis
- 